

```
math_saa/api/chapter_summary.py
```

```
# api/chapter_summary.py
```

```
import os
```

```
import json
```

```
from openai import OpenAI
```

```
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

```
def chapter_summary_handler(payload: dict):
```

```
    class_name = payload.get("class")
```

```
    subject = payload.get("subject")
```

```
    chapter = payload.get("chapter")
```

```
    language = payload.get("language", "English")
```

```
    level = payload.get("level", "basic")
```

```
    prompt = f"""
```

```
    Create a {level} summary for:
```

```
    Class: {class_name}
```

```
    Subject: {subject}
```

```
    Chapter: {chapter}
```

```
    Language: {language}
```

```
    Include:
```

```
    - summary
```

```
    - key_formulas (array)
```

- common_mistakes (array)

Return JSON only.

```
"""
```

```
response = client.chat.completions.create(  
    model="gpt-4o-mini",  
    messages=[{"role": "user", "content": prompt}],  
    response_format={"type": "json_object"}  
)
```

```
# Pylance-safe fix
```

```
raw = response.choices[0].message.content or "{}"  
return json.loads(raw)
```

math_saa/api/generate_questions.py

```
import os  
  
import json  
  
from openai import OpenAI  
  
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))  
  
def generate_questions_handler(payload: dict):  
    class_name = payload.get("class")  
    subject = payload.get("subject")
```

```
chapter = payload.get("chapter")
question_type = payload.get("question_type", "mixed")
count = payload.get("count", 10)
difficulty = payload.get("difficulty", "mixed")
```

```
prompt = f"""
```

```
Generate {count} {difficulty} {question_type} questions for:
```

```
Class: {class_name}
```

```
Subject: {subject}
```

```
Chapter: {chapter}
```

```
Return JSON with fields:
```

- id
- statement
- answer
- solution_steps (array)
- difficulty
- marks

```
"""
```

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"}
)
```

```
# return json.loads(response.choices[0].message.content)
raw = response.choices[0].message.content or "{}"
return json.loads(raw)
```

math_saa/api/main.py

```
from fastapi import FastAPI
```

```
from generate_questions import generate_questions_handler
```

```
from solve_stepwise import solve_stepwise_handler
```

```
from chapter_summary import chapter_summary_handler
```

```
app = FastAPI()
```

```
@app.post("/generate-questions")
```

```
async def generate_questions(payload: dict):
```

```
    return generate_questions_handler(payload)
```

```
@app.post("/solve-stepwise")
```

```
async def solve_stepwise(payload: dict):
```

```
    return solve_stepwise_handler(payload)
```

```
@app.post("/chapter-summary")
```

```
async def chapter_summary(payload: dict):
```

```
    return chapter_summary_handler(payload)
```

math_saa/api/solve_stepwise.py

```
import os
```

```

import json

from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def solve_stepwise_handler(payload: dict):
    question_id = payload.get("question_id")
    student_answer = payload.get("student_answer", "")

    prompt = f"""
    A student answered this question:

    Question ID: {question_id}
    Student Answer: {student_answer}

    Evaluate correctness.

    Provide:

    - is_correct (true/false)
    - correct_answer
    - feedback
    - solution_steps (array)

    Return JSON only.
    """

    response = client.chat.completions.create(

```

```

    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"}
)

# return json.loads(response.choices[0].message.content)
raw = response.choices[0].message.content or "{}"
return json.loads(raw)

```

math_saa/services/question_generator.py

```

import json

from utils.model_router import ask_model

def generate_questions(chapter: str, count: int = 10) -> list:
    """
    Generate exam-style math questions using the intelligent model router.
    Ensures:
    - No Cloudflare 524 timeouts
    - llama3 used for long prompts
    - JSON output is validated and repaired if needed
    """

```

```

# Safety: trim extremely long chapter text
max_chars = 3000
if len(chapter) > max_chars:
    chapter = chapter[:max_chars]

```

```
prompt = f"""
```

You are a math question generator.

Chapter/topic: {chapter}

Generate {count} high-quality math questions of mixed difficulty.

Return STRICTLY valid JSON (no commentary, no markdown), in this format:

```
[  
  {"question": "...", "answer": "...", "difficulty": "easy|medium|hard"},  
  ...  
]  
"""
```

```
# Use the new router (NOT ask_model_math)
```

```
raw = ask_model(prompt, task="questions")
```

```
# Try to parse JSON directly
```

```
try:
```

```
    data = json.loads(raw)
```

```
    if isinstance(data, list):
```

```
        return data
```

```
except Exception:
```

```
    pass
```

```
# Attempt to repair common JSON issues
```

```
repaired = _attempt_json_repair(raw)
```

```
if repaired:
```

```
    return repaired
```

```
# Final fallback: return raw text as a single question
```

```
return [{
```

```
    "question": raw.strip(),
```

```
    "answer": "",
```

```
    "difficulty": "unknown"
```

```
}]
```

```
def _attempt_json_repair(text: str) -> list | None:
```

```
    """
```

```
    Attempts to fix common JSON issues:
```

```
- trailing commas
```

```
- missing brackets
```

```
- extra text before/after JSON
```

```
- single quotes instead of double quotes
```

```
    """
```

```
try:
```

```
    # Extract JSON-like content
```

```
    start = text.find("[")
```

```
    end = text.rfind("]") + 1
```

```
    if start != -1 and end != -1:
```



```

        snippet = text[start:end]

        # Replace single quotes with double quotes
        snippet = snippet.replace("'", '"')

        # Remove trailing commas
        snippet = snippet.replace(",]", "}")

        data = json.loads(snippet)

        if isinstance(data, list):
            return data
        except Exception:
            return None

    return None

```

math_saas/services/solver.py

services/solver.py

import json

from utils.model_router import ask_model

def solve_stepwise(question: str) -> dict:

"""

Step-by-step math solver using the intelligent model router.

- Uses deepseek-r1 only for short prompts
- Uses llama3 for long prompts (avoids Cloudflare 524)
- Ensures JSON output is parsed or safely repaired

```
"""
```

```
# Safety: trim very long questions
```

```
max_chars = 1500
```

```
if len(question) > max_chars:
```

```
    question = question[:max_chars]
```

```
prompt = f"""
```

```
You are an expert math tutor.
```

```
Solve this math problem step-by-step.
```

```
Return STRICT JSON only (no extra text), in this format:
```

```
{{
```

```
  "steps": ["step 1 ...", "step 2 ..."],
```

```
  "final_answer": "..."
```

```
}}
```

```
Problem: {question}
```

```
"""
```

```
# Use the new router (NOT ask_ollama_math)
```

```
raw = ask_model(prompt, task="math")
```

```
# Try direct JSON parse
```

```
try:
```

```
    data = json.loads(raw)
```

```
    if isinstance(data, dict):
```

```
        return data
```

```
except Exception:
```

```
    pass
```

```
# Try to repair malformed JSON
```

```
repaired = _attempt_json_repair(raw)
```

```
if repaired:
```

```
    return repaired
```

```
# Final fallback: wrap raw text
```

```
return {
```

```
    "steps": [raw.strip()],
```

```
    "final_answer": ""
```

```
}
```

```
def _attempt_json_repair(text: str) -> dict | None:
```

```
    """
```

```
    Attempts to fix malformed JSON:
```

```
- extract JSON object
```

```
- replace single quotes
```

```
- remove trailing commas
```

```
    """
```

```
try:
    start = text.find("{")
    end = text.rfind("}") + 1
    if start != -1 and end != -1:
        snippet = text[start:end]

        snippet = snippet.replace("'", "")
        snippet = snippet.replace(",}", "{")

        data = json.loads(snippet)
        if isinstance(data, dict):
            return data
except Exception:
    return None

return None
```

math_saas/services/summary.py

```
# services/summary.py
```

```
from utils.model_router import ask_model
```

```
def summarize_chapter(text: str) -> str:
```

```
"""
```

Summarize a chapter into clear, structured notes.

Uses the router so long prompts go to llama3 and avoid timeouts.

```
"""
```

```
# Safety: trim extremely long input
```

```
max_chars = 4000
```

```
if len(text) > max_chars:
```

```
    text = text[:max_chars]
```

```
prompt = (
```

```
    "You are an expert CBSE math teacher.\n"
```

```
    "Summarize the following chapter into clear, structured notes with headings and bullet  
points.\n\n"
```

```
    f"{text}"
```

```
)
```

```
return ask_model(prompt, task="summary")
```

```
math_saas/utils/config.py
```

```
USE_OLLAMA = "True"
```

```
OLLAMA_URL = "https://ollama.jsdmath.in/api/generate"
```

```
OLLAMA_MODEL_MATH = "deepseek-r1:1.5b"
```

```
OLLAMA_MODEL_SUMMARY = "deepseek-r1:1.5b"
```

```
math_saas/utils/ollama_client.py
```

```
# utils/ollama_client.py
```

```
import requests
```

```
import time
```

```
from typing import Optional
```

```
from utils.config import (
```

```
    OLLAMA_URL,
```

```
    OLLAMA_MODEL_MATH,
```

```
    OLLAMA_MODEL_SUMMARY,
```

```
)
```

```
# Cloudflare can drop long requests → keep timeout high
```

```
DEFAULT_TIMEOUT = 240
```

```
# Retry settings
```

```
MAX_RETRIES = 2
```

```
RETRY_DELAY = 2 # seconds
```

```
def _call_ollama(model: str, prompt: str) -> str:
```

```
    """
```

```
    Robust Ollama caller with:
```

- Retry logic
- Cloudflare-safe timeouts
- Graceful fallback

- Clean error messages

"""

```
payload = {  
    "model": model,  
    "prompt": prompt,  
    "stream": False,  
}
```

```
last_error: Optional[str] = None
```

```
for attempt in range(1, MAX_RETRIES + 1):
```

```
    try:
```

```
        resp = requests.post(  
            OLLAMA_URL,  
            json=payload,  
            timeout=DEFAULT_TIMEOUT,  
        )
```

```
        resp.raise_for_status()
```

```
        data = resp.json()
```

```
        return (data.get("response") or "").strip()
```

```
    except Exception as e:
```

```
        last_error = str(e)
```

```
# Retry only if attempts remain
```

```
if attempt < MAX_RETRIES:
```

```
    time.sleep(RETRY_DELAY)
```

```
    continue
```

```
# Final fallback error
```

```
return f'Ollama error: {last_error}'
```

```
def ask_ollama_math(prompt: str) -> str:
```

```
    """Call the math/logic model."""
```

```
    return _call_ollama(OLLAMA_MODEL_MATH, prompt)
```

```
def ask_ollama_summary(prompt: str) -> str:
```

```
    """Call the fast text model."""
```

```
    return _call_ollama(OLLAMA_MODEL_SUMMARY, prompt)
```